

Building Responsive Layouts with Bootstrap and Tailwind

Modern Website Development

Design mobile-friendly web pages using Bootstrap grid and Tailwind utility classes.



Responsive Design
Looks great on all devices



Flexible Layouts
Grid system and utility classes



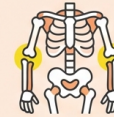
Faster Development
Build modern UIs with ease



Responsive Websites. Real-World Ready.

1

The Web Trinity: HTML vs. CSS vs. JavaScript



HTML

โครงสร้างและเนื้อหา

```
<h1>
  <h1>Hello</h1>
</h1>
```



CSS

หน้าตา สี และการจัดวาง

```
h1 {
  color: red;
}
```



JavaScript

สมอง, พฤติกรรม, การโต้ตอบ

```
document.querySelector(
  'h1').innerText =
  'Hi!';
```

JavaScript คือ 'สมอง' ที่สั่งการให้เว็บทำงานได้

© NotebookLM

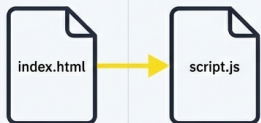
Setup: How to Run JavaScript

1. Internal (เขียนในไฟล์ HTML)



```
<script>
  alert('Hello JavaScript!');
</script>
```

2. External (แยกไฟล์ .js - แนะนำสำหรับงานจริง)



```
<script src='script.js'></script>
```

```
console.log('Hello from external file');
```

Pro Tip: ใช้ Console (F12) ใน Browser เพื่อดูผลลัพธ์และ Debug

© NotebookLM

Variables: Var, Let, Const

ตัวแปรใช้เก็บข้อมูล (Containers for data)



var

Function Scope

Redeclare ได้ (ไม่แนะนำ) ⚠



let

Block Scope ✓

Reassign ได้ (ค่าเปลี่ยนได้)



const

Block Scope ✓

ห้ามเปลี่ยนค่า (ค่าคงที่)

```
JS
// 1. Legacy (Avoid)
var x = 10;

// 2. Modern: Changeable
let score = 0;
score = 10; // OK

// 3. Modern: Constant
const PI = 3.14;
// PI = 3.15; // Error! Assignment to
// constant variable.
```

© NotebookLM

Primitive Data Types

JavaScript เป็น Dynamic Types (เปลี่ยนชนิดข้อมูลได้)

- **String**: ข้อความ ('Alice', 'Hello')
- **Number**: ตัวเลข (จำนวนเต็มและทศนิยม)
- **Boolean**: ค่าความจริง (true / false)
- **Null**: ว่างแบบตั้งใจ (No value)
- **Undefined**: ยังไม่กำหนดค่า

```
let name = 'Alice'; // String
let age = 25; // Number
let isStudent = true; // Boolean
let unknown; // Undefined
let empty = null; // Null

console.log(typeof name); // Output: string
```

Non-Primitive Data Types

Object (เก็บข้อมูลหลายค่า - Key: Value)

```
name: 'John'
age: 30
```

```
let person = {
  name: 'John',
  age: 30
};
console.log(person.name); // John
```

Array (เก็บข้อมูลเป็นลำดับ - Index เริ่มที่ 0)

```
[0] [1] [2] [3]
10  20  30  40
```

```
let numbers = [10, 20, 30, 40];
console.log(numbers[0]); // 10
```

Operators: Math & Logic

Arithmetic Math & Assignment

+ - * / % (Modulus)



```
let x = 10;
x += 5; // x is 15
```

Comparison (ระวัง! Strict vs Loose)

== (Value only) vs === (Value + Type)

```
console.log(5 == '5'); // true
console.log(5 === '5'); // false
```

Logical: Logic Gates

&& (AND), || (OR), ! (NOT)



```
let check = (10 > 5) && (2 < 4); // true
```

Type Conversion

การแปลงชนิดข้อมูล: ปล่อยให้ระบบทำ vs สิ่งทำเอง

Danger Zone

Implicit (Automatic)

JavaScript แปลงให้เอง (ระวังผลลัพธ์ผิดพลาด)

```
let result = '5' + 2;
console.log(result);
// Output: '52' (String)
```

Safe Zone

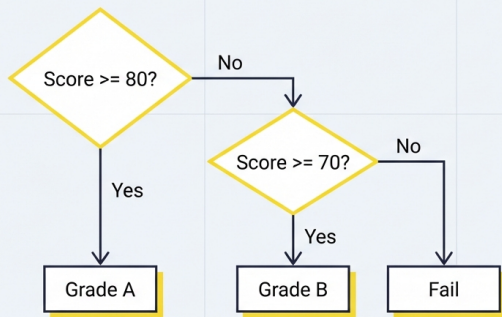
Explicit (Manual)

ผู้เขียนสั่งแปลงเอง (ชัดเจนกว่า)

```
let num = Number('5') + 2;
console.log(num);
// Output: 7 (Math)
```

Control Flow: If / Else

การตัดสินใจของโปรแกรม (Decision Making)



```
JS
let score = 75;

if (score >= 80) {
  console.log('Grade A');
} else if (score >= 70) {
  console.log('Grade B');
} else {
  console.log('Fail');
}
```

© NotebookLM

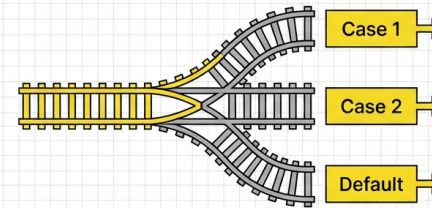
Control Flow: Switch Statement

เลือก 1 ทางจากหลายทาง (Cleaner than many 'else ifs')

Theory

When to use Switch?

เหมาะสำหรับตรวจสอบค่าที่แน่นอน (Exact values)



⚠ อย่าลืม break; เพื่อหยุดการทำงาน

Practice

```
JS
let day = 2;
switch (day) {
  case 1:
    console.log('Monday');
    break;
  case 2:
    console.log('Tuesday');
    break;
  default:
    console.log('Invalid Day');
}
```

© NotebookLM

Loops: For Loop

วนซ้ำแบบรู้จำนวนรอบที่แน่นอน

Theory

```
for ( [Start]; [Condition]; [Step] )
```

1. เริ่มต้น (Init)

2. ตรวจสอบเงื่อนไข (Condition)

3. เพิ่ม/ลดค่า (Increment)

Practice

```
JS
// Count 1 to 5
for (let i = 1; i <= 5; i++) {
  console.log('Round: ' + i);
}
```

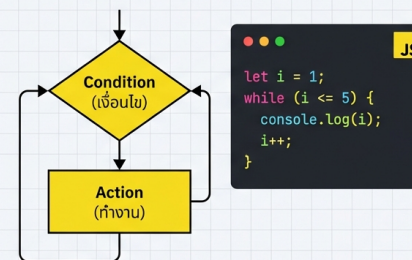
Round: 1
Round: 2
...
Round: 5

© NotebookLM

Loops: While & Do-While

While Loop

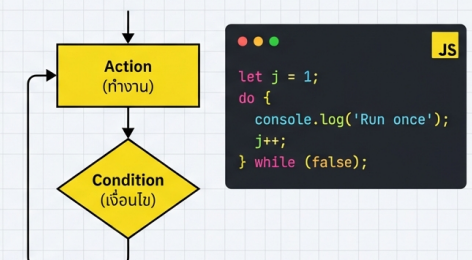
ตรวจสอบเงื่อนไขก่อน (Check first)



```
JS
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

Do-While Loop

ทำงานก่อน 1 ครั้ง แล้วค่อยตรวจสอบ (Run at least once)



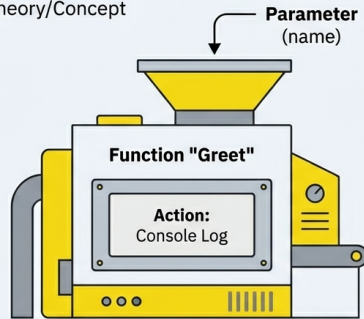
```
JS
let j = 1;
do {
  console.log('Run once');
  j++;
} while (false);
```

© NotebookLM

Functions: Basics

Code ที่เรียกใช้ซ้ำได้ (Reusable Blocks)

Theory/Concept



ประกาศครั้งเดียว เรียกใช้กี่ครั้งก็ได้

Practice

```
// 1. Define Function
function greet(name) {
  console.log('Hello ' + name);
}

// 2. Call Function
greet('Alice');
greet('Bob');
```

JS

© NotebookLM

Return & Scope

Return

ส่งค่ากลับออกจากฟังก์ชัน (Return Value)

```
function add(a, b) {
  return a + b;
}
let sum = add(5, 10); // sum = 15
```

JS

Scope

Local Variable (เรียกใช้จากข้างนอกไม่ได้)



```
function test() {
  let secret = 'Hidden';
}
// console.log(secret); // Error!
```

JS

© NotebookLM

Summary & Key Takeaways

Recap

- ✓ Structure: HTML
- ✓ Style: CSS
- ✓ Brain: JavaScript

Syntax Checklist

- ✓ Variables: let, const (Avoid var)
- ✓ Data Types: String, Number, Boolean, Object, Array
- ✓ Logic: if/else, switch
- ✓ Loops: for, while
- ✓ Functions: parameters, return

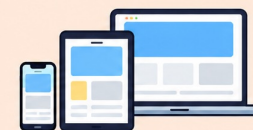
ฝึกฝนผ่าน Console บ่อยๆ เพื่อความชำนาญ!



© NotebookLM

Responsive Layout

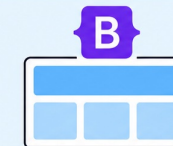
Bootstrap & Tailwind for Modern Web Pages



Responsive Web Design

เว็บปรับตัวตามขนาดหน้าจอ

```
<meta name="viewport"
content="width=device-width,
initial-scale=1.0">
```



Bootstrap

Grid • Utilities • Components

```
<div class="container">
  <div class="row">
    <div class="col-md-4">
      Content
    </div>
  </div>
</div>
```



Tailwind CSS

Utility-First • Flex/Grid • Responsive Prefixes

```
<div class="grid grid-cols-1
md:grid-cols-3 gap-4">
  <div class="bg-blue-100
p-4 rounded-lg">
    Content
  </div>
</div>
```

เป้าหมาย: สร้างหน้าเว็บที่สวยงาม ใช้งานง่าย และตอบสนองทุกอุปกรณ์

Page 1 / 10

Responsive Web Design / Mobile First

เว็บไซต์ที่สวยงามทุกหน้าจอ เริ่มจากมือถือก่อน



Fluid layout

ใช้ %, vw, flex, grid



Flexible media

```
img {
  max-width: 100%;
  height: auto;
}
```



Media queries

ปรับ layout ตาม breakpoint



Mobile First

เริ่มจากจอเล็ก แล้วค่อยเพิ่มที่จอใหญ่

```
.grid { display: grid; gap: 1rem; }
@media (min-width: 768px) {
  .grid { grid-template-columns: 2fr 1fr; }
}
```

Start small, then scale up.

Page 2 / 10

© NotebookLM

Bootstrap Grid

ระบบตาราง 12 คอลัมน์สำหรับการจัดเลย์เอาต์ (Layout)

ระบบ 12 คอลัมน์



- ✓ 12-column system
- ✓ ใช้ .container > .row > .col-*
- ✓ มือถือซ้อนกัน จอใหญ่เรียงแนวนอน

ตัวอย่างโค้ด

8 / 4

```
<div class="container">
  <div class="row g-3">
    <div class="col-12 col-md-8">Main</div>
    <div class="col-12 col-md-4">Sidebar</div>
  </div>
</div>
```

Main
8 columns (md+)

Sidebar
4 columns (md+)

คิดเป็นแถว (row) และคอลัมน์ (column)

Page 3 / 10

Bootstrap: Breakpoints, Utilities, Components



Breakpoints

ขนาดหน้าจอ (Responsive)

ขนาด	ความกว้างขั้นต่ำ (px)
xs	<576
sm	≥576
md	≥768
lg	≥992
xl	≥1200
xxl	≥1400



Utilities

คลาสช่วยจัดรูปแบบ (พร้อมใช้งาน)

.d-flex .justify-content-between
.p-3 .text-center
.d-none .d-md-block



Components

องค์ประกอบสำเร็จรูป (พร้อมใช้งาน)



Navbar



Card



Button



Accordion

```
<div class="d-flex flex-column flex-md-row gap-3 p-3">
  <button class="btn btn-primary">Save</button>
  <div class="card flex-fill">...</div>
</div>
```

Utilities + Components = ทำงานเร็ว

Page 4 / 10

© NotebookLM

Bootstrap Mini Lab

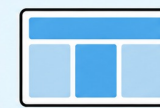
Page 5 / 10



1. Setup

เชื่อมต่อ Bootstrap (CSS)

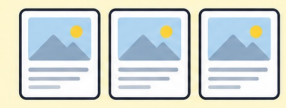
```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet">
```



2. Layout

สร้างโครงสร้าง Grid

```
<div class="container py-4">
  <div class="row g-4">
```



3. Cards

สร้างการ์ดในแต่ละคอลัมน์

```
<div class="col-12 col-md-6 col-lg-4">
  <div class="card h-100 p-3">
    Product
  </div>
</div>
```

มือถือ 1 คอลัมน์ • md 2 คอลัมน์ • lg 3 คอลัมน์

ลองสร้าง responsive cards ด้วย Bootstrap

© NotebookLM

Tailwind CSS: Utility-First



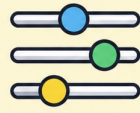
เขียน class เล็ก ๆ
เพื่อประกอบ UI

```
<div class="flex items-center gap-2">
  <span class="px-3 py-1 bg-blue-500 text-white rounded-full">Button</span>
</div>
```



ไม่ต้องตั้งชื่อ CSS เอะ

```
/* .card {
  padding: 24px;
  border-radius: 16px;
  box-shadow: ...
} */
```



แก้ spacing, size, color
ได้ทันทีใน HTML

```
<div class="rounded-2xl bg-white p-6 shadow-md">
  <h2 class="text-xl font-semibold">Card</h2>
</div>
```

Compose styles directly in HTML

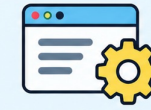
Tailwind: Responsive Prefixes, Flex/Grid



Responsive Prefixes

ปรับหน้าตาตามขนาดหน้าจอ

(ไม่มี)	unprefixed = mobile
sm:	≥ 640px (small)
md:	≥ 768px (medium)
lg:	≥ 1024px (large)
xl:	≥ 1280px (extra large)
2xl:	≥ 1536px (2x large)

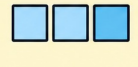


ใช้ class ปกติสำหรับ mobile
แล้วค่อย override ด้วย prefix



Grid

จัดเลย์เอาต์แบบตาราง



Flex

จัดวางองค์ประกอบ
แบบยืดหยุ่น

ตัวอย่างการใช้งาน

```
<div class="grid grid-cols-1 md:grid-cols-3 gap-4">...</div>
<div class="flex flex-col lg:flex-row gap-4">...</div>
```

ตัวอย่าง class เดี่ยว

```
text-center md:text-left
```

Mobile-first เช่น text-center md:text-left

Tailwind Mini Lab

สร้างหน้าเว็บสวย ๆ ด้วย Tailwind CSS แบบเร็ว ๆ

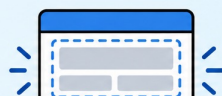


1. Setup

ใส่ meta tag และ CDN ของ Tailwind

```
<meta name="viewport"
  content="width=device-width,
  initial-scale=1.0" />

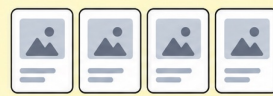
<script src="https://cdn.jsdelivr.net/npm/tailwindcss/browser@4">
</script>
```



2. Wrapper

สร้าง container ตรงกลางหน้า

```
<section class="max-w-6xl mx-auto p-6">
  <!-- เนื้อหาภายใน -->
</section>
```



3. Responsive Grid

จัดวางการ์ดด้วย Grid และ Responsive

```
<div class="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-4 gap-6">
  <div class="rounded-2xl bg-white p-5 shadow">...</div>
</div>
```

mobile 1 card • md 2 cards • xl 4 cards

ทดลองทำ product grid แบบ responsive

Bootstrap vs Tailwind



Bootstrap



Component-based



เริ่มเร็ว มี component พร้อมใช้



เหมาะกับ dashboard /
prototype / งานทีม



Tailwind



Utility-first



ยืดหยุ่นสูง คุณค่าดีไซน์ละเอียด



เหมาะกับ custom UI /
design-heavy projects



ทั้งสองตัวทำ Responsive Layout ได้ดี

เลือก Bootstrap เมื่ออยากเริ่มเร็ว • เลือก Tailwind เมื่ออยากคุมดีไซน์ละเอียด

Summary & Key Takeaways

1



Mobile First
เริ่มจากจอเล็กก่อน

2



Use Grid/Flex
เพื่อจัด layout

3



Bootstrap =
Grid + Utilities +
Components

4



Tailwind =
Utility classes +
Responsive prefixes

5



ฝึกสร้าง layout
เดียวกันทั้งสอง
framework



สิ่งที่สำคัญที่สุด:

Responsive design = readability + usability + adaptability

Remember: layout first, polish second.

Page 10 / 10